

Introduction to Software Quality Assurance

using

Visual Studio, GitHub, and GitHub Copilot

By the end of this lab, students should be able to:

- Set up a basic development environment using Visual Studio and GitHub
- Explain the difference between QA and QC using a practical coding example
- Identify functional and non-functional quality issues in a small software system
- Use GitHub Copilot to support test idea generation and code review
- Commit and push lab work to GitHub as evidence of progress, as expected in ENSE707

Problem Statement

A small community bank wants a simple Bank Account and Payment Calculator application to help staff demonstrate basic customer account operations. The application should allow a staff member to create a customer account, record deposits, process withdrawals, prevent customers from withdrawing more money than they have available, calculate a transaction fee, and display the customer's current account balance. Although this is only a small prototype, the bank expects the calculations to be accurate, invalid transactions to be handled correctly, and the code to be easy to understand, test, and improve. The purpose of this lab is to use this simple banking scenario to explore software quality issues such as correctness, validation, reliability, maintainability, and testability.

Lab Scenario

Please work with a small Bank Account / Payment Calculator application. The application should:

- Create a customer account
- Deposit money
- Withdraw money
- Prevent overdrafts

- Calculate a transaction fee
- Display account balance

The system is intentionally simple so you can focus on software quality, not complex programming.

Activity 1 — Environment and Repository Setup

Tasks

- i. Open Visual Studio.
- ii. Create a new project:
 - a. Project type: Console App
 - b. Language: C#
 - c. Project name: ENSE707_Week1_Lab
- iii. Create a GitHub repository with the same name.
- iv. Connect the Visual Studio project to GitHub.
- v. Make an initial commit:
 - a. Commit message: Initial Week 1 lab project setup

Write answers for the following questions:

1. Why does regular committing support software quality?
2. How can Git history provide evidence of individual progress?

Activity 2 — Add the Starter Code

Create a class called BankAccount.cs.

```
namespace ENSE707_Week1_Lab;

public class BankAccount
{
    public string AccountHolder { get; set; }
    public decimal Balance { get; private set; }

    public BankAccount(string accountHolder, decimal openingBalance)
    {
        AccountHolder = accountHolder;
        Balance = openingBalance;
    }

    public void Deposit(decimal amount)
    {
        Balance = Balance + amount;
    }

    public bool Withdraw(decimal amount)
    {
        Balance = Balance - amount;
        return true;
    }

    public decimal CalculateTransactionFee(decimal amount)
    {
        return amount * 0.02m;
    }
}
```

Then update Program.cs

```
using ENSE707_Week1_Lab;

BankAccount account = new BankAccount("Student User", 100);

account.Deposit(50);
account.Withdraw(30);

Console.WriteLine($"Account holder: {account.AccountHolder}");
Console.WriteLine($"Current balance: {account.Balance}");
Console.WriteLine($"Fee on $100 transaction: {account.CalculateTransactionFee(100)}");
```

Task

Run the program and observe the output.

Activity 3 — Identify Quality Issues

Please inspect the code and answer the following questions:

1. What happens if the opening balance is negative?
2. What happens if a deposit amount is negative?
3. What happens if a withdrawal amount is greater than the balance?
4. Is the transaction fee calculation clearly documented?
5. Is the class easy to test?
6. What functional requirements are missing?
7. What non-functional quality attributes are relevant?

Activity 4 — QA vs QC

Write a comprehensive note comparing Quality Assurance (QA) and Quality Control (QC). Then, classify each of the following activities as either QA or QC, providing a brief justification for your answer.

Activities

- Writing coding standards for money calculations
- Running unit tests for withdrawal behaviour
- Reviewing requirements for ambiguity
- Testing negative deposit input
- Analysing repeated transaction defects
- Reporting a failed test case
- Creating a checklist for financial validation rules
- Retesting after fixing withdrawal logic

Discussion Question

Why is writing tests not enough to guarantee quality?

Activity 5 — Use Copilot for Test Ideas

You may ask GitHub Copilot:

- *Suggest unit test cases for a C# BankAccount class with deposit, withdraw, and transaction fee methods.*
- *Focus on boundary values and invalid inputs.*

Important Instruction

Do not blindly accept the output generated by GitHub Copilot or any other AI-assisted tool. Carefully review all suggested tests to ensure they are:

- Relevant to the requirements
- Correct and logically valid
- Complete, covering appropriate scenarios
- Understandable and well-structured
- Aligned with the specified requirements
- Safe and appropriate to include in your submission

This aligns with the expectations of this course. You are *encouraged to use AI-assisted tools* to support your learning and development, but *you are responsible* for understanding, reviewing, and being able to explain all the code and tests you submit as your own work.

Activity 6 — Create a Unit Test Project

Tasks

1. Add a new project to the solution:
 - a. Project type: MSTest Test Project or xUnit Test Project
 - b. Name: ENSE707_Week1_Lab.Tests
2. Add a project reference to the main console project.
3. Create a test class called BankAccountTests.cs.

Sample using MSTest:

```
using Microsoft.VisualStudio.TestTools.UnitTesting;
using ENSE707_Week1_Lab;

namespace ENSE707_Week1_Lab.Tests;

[TestClass]
public class BankAccountTests
{
    [TestMethod]
    public void Deposit_PositiveAmount_IncreasesBalance()
    {
        BankAccount account = new BankAccount("Test User", 100);

        account.Deposit(50);

        Assert.AreEqual(150, account.Balance);
    }

    [TestMethod]
    public void Withdraw_ValidAmount_DecreasesBalance()
    {
        BankAccount account = new BankAccount("Test User", 100);

        bool result = account.Withdraw(40);

        Assert.IsTrue(result);
        Assert.AreEqual(60, account.Balance);
    }
}
```

```

using Microsoft.VisualStudio.TestTools.UnitTesting;
using ENSE707_Week1_Lab;

namespace ENSE707_Week1_Lab.Tests;

[TestClass]
public class BankAccountTests
{
    [TestMethod]
    public void Deposit_PositiveAmount_IncreasesBalance()
    {
        BankAccount account = new BankAccount("Test User", 100);

        account.Deposit(50);

        Assert.AreEqual(150, account.Balance);
    }

    [TestMethod]
    public void Withdraw_ValidAmount_DecreasesBalance()
    {
        BankAccount account = new BankAccount("Test User", 100);

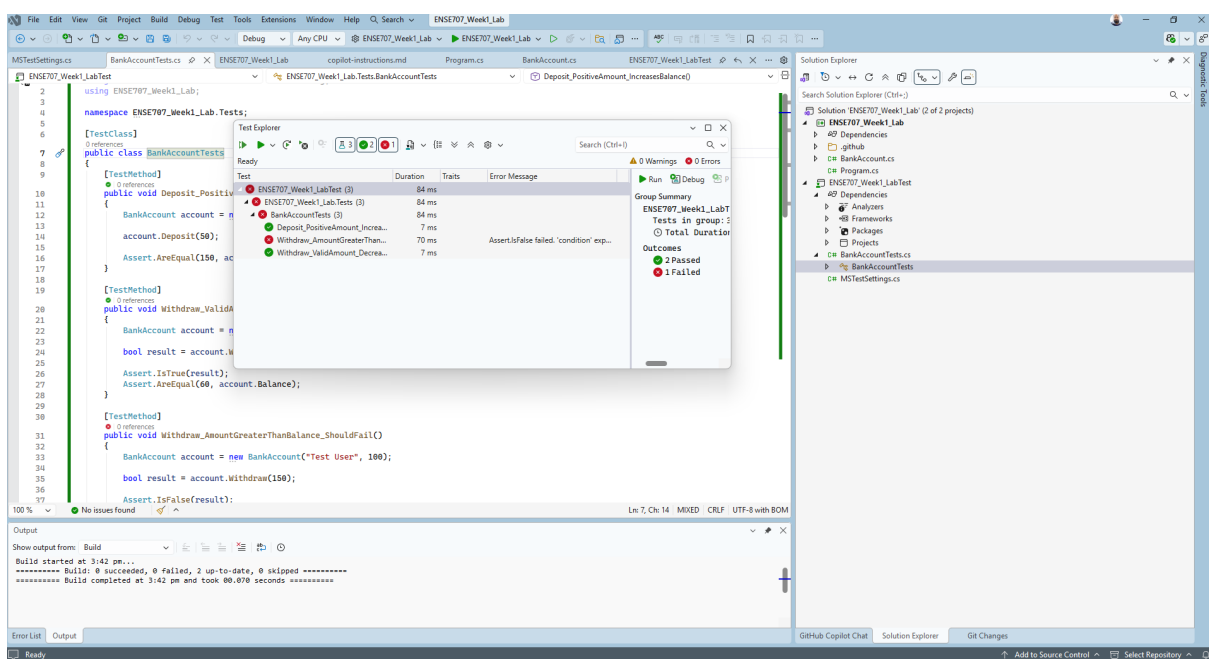
        bool result = account.Withdraw(40);

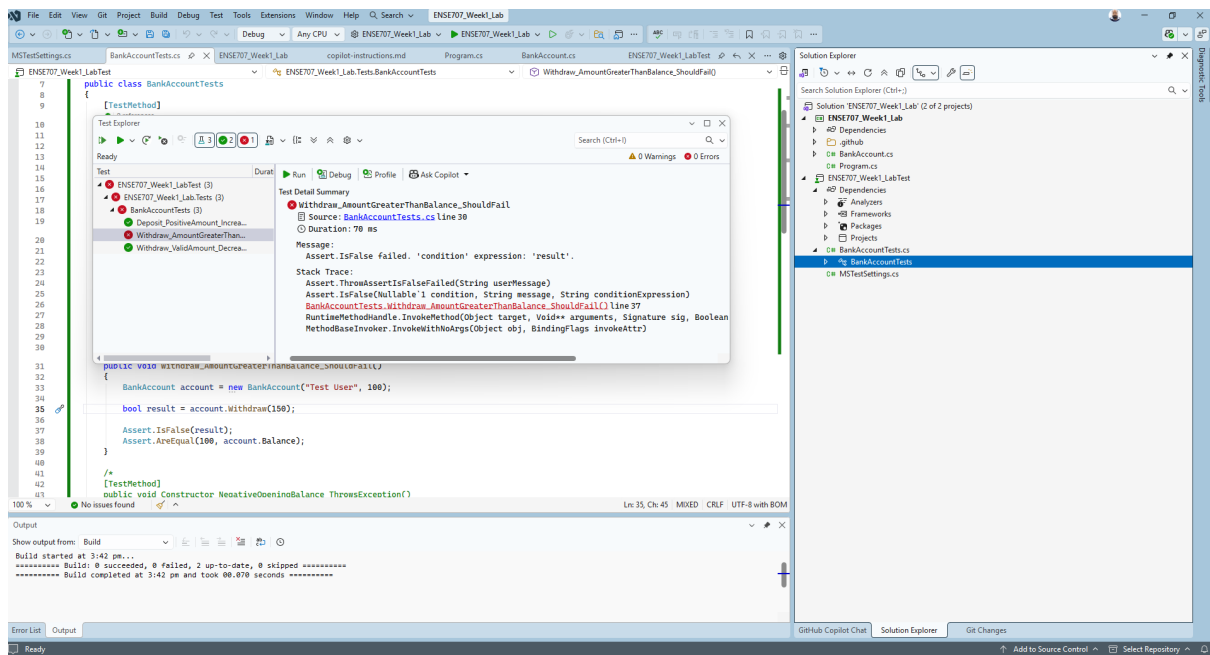
        Assert.IsTrue(result);
        Assert.AreEqual(60, account.Balance);
    }
}

```

Expected Result

At least one test should fail because the starter code allows overdrafts.





Activity 7 — Improve your code

Improve your main code (BankAccount.cs). Sample improved version:

```
namespace ENSE707_Week1_Lab;

public class BankAccount
{
    public string AccountHolder { get; }
    public decimal Balance { get; private set; }

    public BankAccount(string accountHolder, decimal openingBalance)
    {
        if (string.IsNullOrEmpty(accountHolder))
            throw new ArgumentException("Account holder name is required.");

        if (openingBalance < 0)
            throw new ArgumentException("Opening balance cannot be negative.");

        AccountHolder = accountHolder;
        Balance = openingBalance;
    }

    public void Deposit(decimal amount)
    {
        if (amount <= 0)
            throw new ArgumentException("Deposit amount must be positive.");

        Balance += amount;
    }
}
```



```
public bool Withdraw(decimal amount)
{
    if (amount <= 0)
        throw new ArgumentException("Withdrawal amount must be positive.");

    if (amount > Balance)
        return false;

    Balance -= amount;
    return true;
}

public decimal CalculateTransactionFee(decimal amount)
{
    if (amount <= 0)
        throw new ArgumentException("Transaction amount must be positive.");

    return Math.Round(amount * 0.02m, 2);
}
```

Quality Improvement Discussion

Please identify which quality attributes are improved:

- Correctness
- Reliability
- Maintainability
- Testability
- Security
- User trust

Activity 8 — Add More Tests

Please add at least four additional tests, such as:

```

[TestMethod]
public void Constructor_NegativeOpeningBalance_ThrowsException()
{
    Assert.ThrowsException<ArgumentException>(() =>
        new BankAccount("Test User", -10));
}

[TestMethod]
public void Deposit_NegativeAmount_ThrowsException()
{
    BankAccount account = new BankAccount("Test User", 100);

    Assert.ThrowsException<ArgumentException>(() =>
        account.Deposit(-20));
}

[TestMethod]
public void Withdraw_NegativeAmount_ThrowsException()
{
    BankAccount account = new BankAccount("Test User", 100);

    Assert.ThrowsException<ArgumentException>(() =>
        account.Withdraw(-20));
}

[TestMethod]
public void CalculateTransactionFee_ValidAmount_ReturnsTwoPercentFee()
{
    BankAccount account = new BankAccount("Test User", 100);

    decimal fee = account.CalculateTransactionFee(200);

    Assert.AreEqual(4.00m, fee);
}

```

Activity 9 — Stakeholder and Quality Reflection Questions

1. Who are the stakeholders for this small banking system?
2. What does quality mean to each stakeholder?
3. Which defects were detected through testing?
4. Which defects could have been prevented through QA activities?
5. How did Copilot help?
6. What Copilot suggestion did you reject or modify, and why?

7. What is the difference between QA and QC in this lab?

Activity 10 — Commit and Push

Students must commit at least two times per week.